

API REST NORA

Fecha: 30/07/2025

Referencia: norarest

EJIE S.A.
Mediterráneo, 14
Tel. 945 01 73 00*
Fax. 945 01 73 01
01010 Vitoria-/Gasteiz
Posta-kutxatila / Apartado: 809
01080 Vitoria-Gasteiz
www.ejie.es

Este documento es propiedad de EJIE, S.A. y su contenido es confidencial. Este documento no puede ser reproducido, en su totalidad o parcialmente, ni mostrado a otros, ni utilizado para otros propósitos que los que han originado su entrega, sin el previo permiso escrito de EJIE, S.A.. En el caso de ser entregado en virtud de un contrato, su utilización estará limitada a lo expresamente autorizado en dicho contrato. EJIE, S.A. no podrá ser considerada responsable de eventuales errores u omisiones en la edición del documento.

Contenido		
	Capítulo/sección	Página
1	Introducción	7
2	API REST	8
2.1.1.	Cliente (ejemplos en javascript)	8
2.1.2.	Servidor (ejemplos en Java)	10
3	Anexo I: Entidades	18
3.1	Entidades	18
3.1.1.	País	18
3.1.2.	Autonomía	19
3.1.3.	Provincia	19
3.1.4.	Comarca	20
3.1.5.	Municipio	21
3.1.6.	Entidad	22
3.1.7.	Localidad	22
3.1.8.	Barrio	23
3.1.9.	Distrito	23
3.1.10.	Sección	24
3.1.11.	Calle	25
3.1.12.	Portal	26
3.1.13.	TipoVía	27
3.1.14.	Edificios	28

1 Introducción

Los datos de localizaciones del tipo provincia, municipio, localidad, calle, portal... son informaciones ampliamente incorporadas en multitud de aplicaciones, bases de datos, registros o formularios de todos los Departamentos, órganos y entidades del sector público de la Comunidad Autónoma del País Vasco.

Desde el origen del proyecto NORA se busca la normalización y uso de una base de referencia para todos, así como facilitar la integración de dicha información en estas múltiples aplicaciones.

Un ejemplo ilustrativo de este uso podrían ser los datos relativos a la sede social de una empresa que participa en la tramitación de un expediente para un departamento X. Esto permitirá localizar sobre un mapa todas las empresas, así como agrupar datos por localidad o sección censal, o incluso explotar espacialmente la base de datos combinándola con datos de otra posible base de datos de población para calcular personas que viven a una determinada distancia de una supuesta serie de empresas potencialmente contaminantes.

Eustat mantiene el conjunto de datos denominado “Callejero de la Comunidad Autónoma del País Vasco” que se compone de los Portales Oficiales del País Vasco más aquellos edificios que, si bien no tienen una dirección oficial, son de interés, almacenando el bloque, el número y el portal bis. Además, dispone de una capa de ejes de calles, la cual recoge un sistema reticular que se extiende por todos los viales de la Comunidad Autónoma Vasca comunicando todos los portales entre sí. El mantenimiento de este conjunto de datos requiere un esfuerzo y una constante coordinación con todos los ayuntamientos de la CAE, incluyendo la gestión de altas provisionales o incidencias provenientes de sus múltiples usos en organizaciones como Osakidetza o Industria.

Los objetivos principales del proyecto han sido:

- Establecer un marco de trabajo a nivel de datos con el Eustat con el fin de garantizar que los datos disponibles en Eustat y los ofrecidos estén en concordancia.
- Construir Servicios de valor añadido que puedan ser utilizados de forma horizontal.
- Integrar dichos servicios en el geolocalizador de geoEuskadi, así como con el API geoEuskadi para construir aplicaciones.

Desde la puesta en marcha de la primera solución de acceso a datos de localización sobre el año 2008, se han ido actualizando diariamente los datos alfanuméricos, y trimestralmente los datos geográficos, así como evolucionando los componentes tecnológicos de los servicios NORA.

Actualmente la amplia mayoría de aplicaciones en Gobierno Vasco emplean los servicios NORA para almacenar datos de localización y dicha capacidad permite la geolocalización directa en el territorio de estos datos de localizaciones.

También el geolocalizador de geoEuskadi, así como los diferentes componentes, disponen de capacidades para consumir, consultar y ubicar en el mapa datos provenientes de los servicios NORA.

2 API REST

Se ha definido un API que posibilita interrogar los datos de territorio disponibles. Las funciones y sus correspondientes parámetros son los mismos y lo único que cambia es la forma de invocarlas dependiendo de la tecnología utilizada.

Para obtener más información sobre las entidades ver Anexo I.

Posibilita obtener datos mediante peticiones Web Service de tipo REST.

El API está pensado para obtener datos de localizaciones mediante peticiones estándar Web Services REST sin requerir ninguna librería específica. Las invocaciones son realizadas de forma síncrona.

La información se puede obtener en formato JSON o en formato JSONP para evitar problemas de cross-domain.

2.1.1. Cliente (ejemplos en javascript)

2.1.1.1. Ejemplo 1: Consultas obteniendo un listado de países

En el siguiente ejemplo se muestra el código javascript necesario para obtener un listado sencillo de países recibiendo tanto un resultado de tipo JSON como de un JSONP:

1. Realizar un fetch al método REST del que recuperar el JSON.
2. Llamar al método JSONP y guardar su contenido como fuente de Javascript.
3. Añadir un método con el mismo nombre que el parámetro callback introducido en la llamada a la url.
4. Tratar los resultados. En este caso, sólo se printan en la consola del navegador.

```
// Llamada a un webservice REST que devuelve un JSON.
fetch('https://www.geo.euskadi.eus/t17iApiRestWar/rest/v1/paises?withParents=false')
  .then(response => response.json())
  .then(data => console.log(data))
  .catch(error => console.error(error));

// Llamada a un webservice REST que devuelve un JSONP.
function parseJsonpResponse(jsonpResponse) {
  console.log(jsonpResponse);
}

script = document.createElement('script');
script.src = '
https://www.geo.euskadi.eus/t17iApiRestWar/rest/v1/jsonp/paises?withParents=false&callback=parseJ
sonpResponse';
document.getElementsByTagName('head')[0].appendChild(script);
```

2.1.1.2. Ejemplo 2: Consultas obteniendo un listado de comarca usando callback

En el siguiente ejemplo se muestra el código javascript necesario para obtener información de la comarca filtrando a través de las provincias y recibir la información tanto en formato JSON como de un JSONP utilizando un callback:

1. Realizar un fetch al método REST del que recuperar el JSON.
2. Llamar al método JSONP y guardar su contenido como fuente de Javascript.
3. Añadir un método con el mismo nombre que el parámetro callback introducido en la llamada a la url.
4. Tratar los resultados. En este caso, sólo se printan en la consola del navegador.

```
//Llamada a un webservice REST que devuelve un JSON
fetch('https://www.geo.euskadi.eus/t17iApiRestWar/rest/v1/comarcas/48/02?withParents=false')
  .then(response => response.json())
  .then(data => console.log(data))
  .catch(error => console.error(error));

// Llamada a un webservice REST que devuelve un JSONP.
function parseJsonpResponse(jsonpResponse) {
  console.log(jsonpResponse);
}

script = document.createElement('script');
script.src =
'https://www.geo.euskadi.eus/t17iApiRestWar/rest/v1/jsonp/comarcas/48/02?withParents=false&callba
ck=parseJsonpResponse';
document.getElementsByTagName('head')[0].appendChild(script);
```

2.1.1.3. Ejemplo 3: Consultas obteniendo localidades por coordenadas

En el siguiente ejemplo se muestra el código javascript necesario para obtener un listado sencillo de países recibiendo tanto un resultado de tipo JSON como de un JSONP:

1. Realizar un fetch al método REST del que recuperar el JSON.
2. Llamar al método JSONP y guardar su contenido como fuente de Javascript.
3. Añadir un método con el mismo nombre que el parámetro callback introducido en la llamada a la url.
4. Tratar los resultados. En este caso, sólo se printan en la consola del navegador.

```
//Llamada a un webservice REST que devuelve un JSON
fetch('https://www.geo.euskadi.eus/t17iApiRestWar/rest/v1/localidades/coordenadas?x=511871.0909&
y=4786426.010&crs=EPSG%3A25830')
  .then(response => response.json())
  .then(data => console.log(data))
  .catch(error => console.error(error));
```

```
// Llamada a un webservice REST que devuelve un JSONP.  
function parseJsonpResponse(jsonpResponse) {  
    console.log(jsonpResponse);  
}  
script = document.createElement('script');  
script.src = '  
https://www.geo.euskadi.eus/t17iApiRestWar/rest/v1/jsonp/localidades/coordenadas?x=511871.0909&  
y=4786426.010&crs=EPSG%3A25830&callback=parseJsonpResponse';  
document.getElementsByTagName('head')[0].appendChild(script);
```

2.1.1.4. Ejemplo 4: Consultas con filtro sobre campos opcionales

En este caso hacemos una petición en la que algunos de los campos no son necesarios.

1. Realizar un fetch al método REST del que recuperar el JSON.
2. Llamar al método JSONP y guardar su contenido como fuente de Javascript.
3. Añadir un método con el mismo nombre que el parámetro callback introducido en la llamada a la url.
4. Tratar los resultados. En este caso, sólo se printan en la consola del navegador.

```
//Llamada a un webservice REST que devuelve un JSON  
fetch(' https://www.geo.euskadi.eus  
/t17iApiRestWar/rest/v1/municipios?descMunicipio=Bilbao&withParents=false')  
    .then(response => response.json())  
    .then(data => console.log(data))  
    .catch(error => console.error(error));  
  
// Llamada a un webservice REST que devuelve un JSONP.  
function parseJsonpResponse(jsonpResponse) {  
    console.log(jsonpResponse);  
}  
  
script = document.createElement('script');  
script.src = ' https://www.geo.euskadi.eus  
/t17iApiRestWar/rest/v1/jsonp/municipios?descMunicipio=Bilbao&withParents=false&callback=parseJ  
sonpResponse';  
document.getElementsByTagName('head')[0].appendChild(script);
```

2.1.2. Servidor (ejemplos en Java)

2.1.2.1. Ejemplo 1 (Consultas obteniendo un listado de países)

En el siguiente ejemplo se muestra el código java necesario para obtener un listado sencillo de países recibiendo tanto un resultado de tipo JSON como de un JSONP:

1. Añadir la librería Gson al proyecto.
2. Crear la clase controlador de SPRING que realizará la llamada al servicio REST.
3. Realizar la llamada al servicio REST utilizando, por ejemplo, las clases `java.net.HttpURLConnection` y `java.io.BufferedReader` para almacenar el resultado recibido.
4. Transformar en un objeto JSON el resultado recibido utilizando la librería Gson.
5. Tratar los resultados. En este caso, sólo se printan en consola.

```
import java.io.BufferedReader;
import java.io.IOException;
import java.io.InputStreamReader;
import java.net.HttpURLConnection;
import java.net.URL;
import java.util.ArrayList;
import java.util.List;

import org.springframework.http.MediaType;
import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestMethod;

import com.google.gson.JsonArray;
import com.google.gson.JsonElement;
import com.google.gson.JsonObject;
import com.google.gson.JsonParser;

@Controller
@RequestMapping(value = "/pais")
public class PaisController {
    private static final String API_URL_JSON = "https://www.geo.euskadi.eus/t17iApiRestWar/rest/v1/paises?withParents=false";
    private static final String API_URL_JSONP = "https://www.geo.euskadi.eus/t17iApiRestWar/rest/v1/jsonp/paises?withParents=false&callback=callback";

    @RequestMapping(value = "/", method = RequestMethod.GET, produces =
    MediaType.APPLICATION_JSON_VALUE)
    public void ejemplo() throws Exception {
        // Formato JSON.
        String jsonResponse = this.callRestService(API_URL_JSON);
        List<JsonObject> dataObjects = this.parseJsonResponse(jsonResponse);
        System.out.println(dataObjects);

        // Formato JSONP.
        jsonResponse = this.callRestService(API_URL_JSONP);
        dataObjects = this.parseJsonpResponse(jsonResponse);
        System.out.println(dataObjects);
    }
}
```

```
}

public String callRestService(String urlString) {
    StringBuilder result = new StringBuilder();
    try {
        URL url = new URL(urlString);
        HttpURLConnection conn = (HttpURLConnection) url.openConnection();
        conn.setRequestMethod("GET");
        BufferedReader rd = new BufferedReader(new
InputStreamReader(conn.getInputStream()));
        String line;
        while ((line = rd.readLine()) != null) {
            result.append(line);
        }
        rd.close();
    } catch (IOException e) {
        e.printStackTrace();
    }
    return result.toString();
}

public List<JsonObject> parseJsonResponse(String jsonResponse) {
    List<JsonObject> dataObjects = new ArrayList<JsonObject>();
    JsonParser parser = new JsonParser();
    JsonElement element = parser.parse(jsonResponse);
    if (element.isJsonArray()) {
        JsonArray jsonArray = element.getAsJsonArray();
        for (JsonElement jsonElement : jsonArray) {
            JsonObject jsonObject = jsonElement.getAsJsonObject();
            dataObjects.add(jsonObject);
        }
    }
    return dataObjects;
}

public List<JsonObject> parseJsonpResponse(String jsonpResponse) {
    String jsonResponse = jsonpResponse.substring(jsonpResponse.indexOf("(") + 1,
jsonpResponse.lastIndexOf(")"));
    return this.parseJsonResponse(jsonResponse);
}
}
```

2.1.2.2. Ejemplo 2: Consultas obteniendo un listado de comarca

```
import java.io.BufferedReader;
import java.io.IOException;
import java.io.InputStreamReader;
import java.net.HttpURLConnection;
import java.net.URL;
```

```
import java.util.ArrayList;
import java.util.List;

import org.springframework.http.MediaType;
import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestMethod;

import com.google.gson.JsonArray;
import com.google.gson.JsonElement;
import com.google.gson.JsonObject;
import com.google.gson.JsonParser;

@Controller
@RequestMapping(value = "/comarca")
public class ComarcaController {
    private static final String API_URL_JSON = " https://www.geo.euskadi.eus
/t17iApiRestWar/rest/v1/comarcas?withParents=false";
    private static final String API_URL_JSONP = " https://www.geo.euskadi.eus
/t17iApiRestWar/rest/v1/jsonp/comarcas?withParents=false&callback=callback";

    @RequestMapping(value = "/", method = RequestMethod.GET, produces =
MediaType.APPLICATION_JSON_VALUE)
    public void ejemplo() throws Exception {
        // Formato JSON.
        String jsonResponse = this.callRestService(API_URL_JSON);
        List<JsonObject> dataObjects = this.parseJsonResponse(jsonResponse);
        System.out.println(dataObjects);

        // Formato JSONP.
        jsonResponse = this.callRestService(API_URL_JSONP);
        dataObjects = this.parseJsonpResponse(jsonResponse);
        System.out.println(dataObjects);
    }

    public String callRestService(String urlString) {
        StringBuilder result = new StringBuilder();
        try {
            URL url = new URL(urlString);
            HttpURLConnection conn = (HttpURLConnection) url.openConnection();
            conn.setRequestMethod("GET");
            BufferedReader rd = new BufferedReader(new
InputStreamReader(conn.getInputStream()));
            String line;
            while ((line = rd.readLine()) != null) {
                result.append(line);
            }
            rd.close();
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

```

        }
        return result.toString();
    }

    public List<JsonObject> parseJsonResponse(String jsonResponse) {
        List<JsonObject> dataObjects = new ArrayList<JsonObject>();
        JsonParser parser = new JsonParser();
        JsonElement element = parser.parse(jsonResponse);
        if (element.isArray()) {
            JsonArray jsonArray = element.getAsJsonArray();
            for (JsonElement jsonElement : jsonArray) {
                JsonObject jsonObject = jsonElement.getAsJsonObject();
                dataObjects.add(jsonObject);
            }
        }
        return dataObjects;
    }

    public List<JsonObject> parseJsonpResponse(String jsonpResponse) {
        String jsonResponse = jsonpResponse.substring(jsonpResponse.indexOf("(") + 1,
            jsonpResponse.lastIndexOf(")"));
        return this.parseJsonResponse(jsonResponse);
    }
}

```

2.1.2.3. Ejemplo 3: Consultas obteniendo localidades por coordenadas

```

import java.io.BufferedReader;
import java.io.IOException;
import java.io.InputStreamReader;
import java.net.HttpURLConnection;
import java.net.URL;
import java.util.ArrayList;
import java.util.List;

import org.springframework.http.MediaType;
import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestMethod;

import com.google.gson.JsonArray;
import com.google.gson.JsonElement;
import com.google.gson.JsonObject;
import com.google.gson.JsonParser;

@Controller
@RequestMapping(value = "/localidad")

```

```

public class LocalidadController {
    private static final String API_URL_JSON = " https://www.geo.euskadi.eus
/t17iApiRestWar/rest/v1/localidades/coordenadas?x=505284.827&y=4789962.337&crs=EPSG%3A2
5830";
    private static final String API_URL_JSONP = " https://www.geo.euskadi.eus
/t17iApiRestWar/rest/v1/jsonp/localidades/coordenadas?x=505284.827&y=4789962.337&crs=EPSG
%3A25830&callback=callback";

    @RequestMapping(value = "/", method = RequestMethod.GET, produces =
MediaType.APPLICATION_JSON_VALUE)
    public void ejemplo() throws Exception {
        // Formato JSON.
        String jsonResponse = this.callRestService(API_URL_JSON);
        List<JsonObject> dataObjects = this.parseJsonResponse(jsonResponse);
        System.out.println(dataObjects);
        // Formato JSONP.
        jsonResponse = this.callRestService(API_URL_JSONP);
        dataObjects = this.parseJsonpResponse(jsonResponse);
        System.out.println(dataObjects);
    }

    public String callRestService(String urlString) {
        StringBuilder result = new StringBuilder();
        try {
            URL url = new URL(urlString);
            HttpURLConnection conn = (HttpURLConnection) url.openConnection();
            conn.setRequestMethod("GET");
            BufferedReader rd = new BufferedReader(new
InputStreamReader(conn.getInputStream()));
            String line;
            while ((line = rd.readLine()) != null) {
                result.append(line);
            }
            rd.close();
        } catch (IOException e) {
            e.printStackTrace();
        }
        return result.toString();
    }

    public List<JsonObject> parseJsonResponse(String jsonResponse) {
        List<JsonObject> dataObjects = new ArrayList<JsonObject>();
        JsonParser parser = new JsonParser();
        JsonElement element = parser.parse(jsonResponse);
        if (element.isJsonArray()) {
            JsonArray jsonArray = element.getAsJsonArray();
            for (JsonElement jsonElement : jsonArray) {
                JsonObject jsonObject = jsonElement.getAsJsonObject();
                dataObjects.add(jsonObject);
            }
        }
    }
}

```

```

        }
        return dataObjects;
    }

    public List<JsonObject> parseJsonpResponse(String jsonpResponse) {
        String jsonResponse = jsonpResponse.substring(jsonpResponse.indexOf("(") + 1,
jsonpResponse.lastIndexOf(")"));
        return this.parseJsonResponse(jsonResponse);
    }
}

```

2.1.2.4. Ejemplo 4 (Consultas con filtro sobre campos opcionales)

```

@Controller
@RequestMapping(value = "/municipio")
public class MunicipioController {

    private static final String API_URL_JSON = " https://www.geo.euskadi.eus
/t17iApiRestWar/rest/v1/municipios?descMunicipio=Bilbao&withParents=false&callback=parseJsonp
Response";

    private static final String API_URL_JSONP = " https://www.geo.euskadi.eus
/t17iApiRestWar/rest/v1/jsonp/municipios?descMunicipio=Bilbao&callback=callback";

    @RequestMapping(value = "/", method = RequestMethod.GET, produces =
MediaType.APPLICATION_JSON_VALUE)
    public void ejemplo() throws Exception {
        // Formato JSON.
        String jsonResponse = this.callRestService(API_URL_JSON);
        List<JsonObject> dataObjects = this.parseJsonResponse(jsonResponse);
        System.out.println(dataObjects);

        // Formato JSONP.
        jsonResponse = this.callRestService(API_URL_JSONP);
        dataObjects = this.parseJsonpResponse(jsonResponse);
        System.out.println(dataObjects);
    }

    public String callRestService(String urlString) {
        StringBuilder result = new StringBuilder();
        try {

```

```

        URL url = new URL(urlString);
        HttpURLConnection conn = (HttpURLConnection) url.openConnection();
        conn.setRequestMethod("GET");
        BufferedReader rd = new BufferedReader(new
InputStreamReader(conn.getInputStream()));
        String line;
        while ((line = rd.readLine()) != null) {
            result.append(line);
        }
        rd.close();
    } catch (IOException e) {
        e.printStackTrace();
    }
    return result.toString();
}

public List<JsonObject> parseJsonResponse(String jsonResponse) {
    List<JsonObject> dataObjects = new ArrayList<JsonObject>();
    JsonParser parser = new JsonParser();
    JsonElement element = parser.parse(jsonResponse);
    if (element.isJsonArray()) {
        JsonArray jsonArray = element.getAsJsonArray();
        for (JsonElement jsonElement : jsonArray) {
            JsonObject jsonObject = jsonElement.getAsJsonObject();
            dataObjects.add(jsonObject);
        }
    }
    return dataObjects;
}

public List<JsonObject> parseJsonpResponse(String jsonpResponse) {
    String jsonResponse = jsonpResponse.substring(jsonpResponse.indexOf("(") + 1,
jsonpResponse.lastIndexOf(")"));
    return this.parseJsonResponse(jsonResponse);
}
}

```

3 Anexo I: Entidades

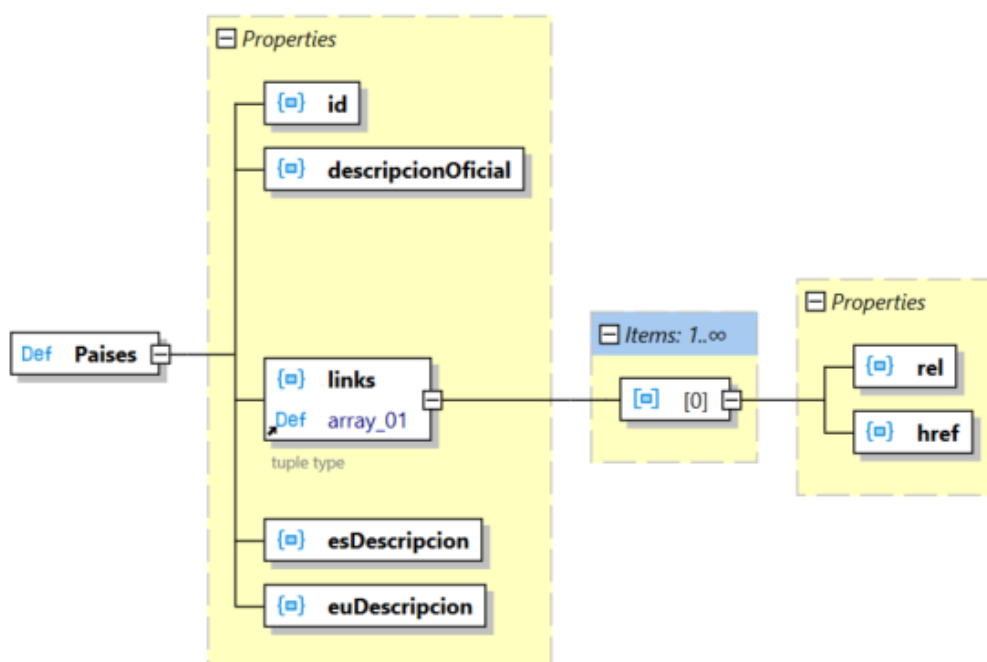
3.1 Entidades

El proyecto maneja 14 entidades de datos de Territorio: país, autonomía, provincia, comarca, municipio, entidad, localidad, barrio, distrito, seccion, calle, portal, tipo de vía, edificio.

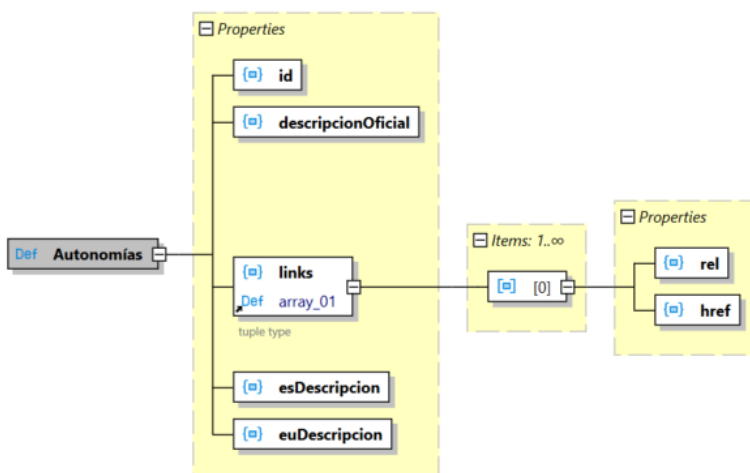
Las Funciones disponibles sobre las entidades definidas posibilitan básicamente buscar en base a descripciones y claves.

En todas las funciones existe la posibilidad de decidir la profundidad de los resultados mediante una propiedad: **withParents**. Por ejemplo, es posible realizar una búsqueda de municipios obteniendo únicamente los datos propios del municipio (id, idProvincia, descripcionOficial) para cada resultado. Pero también es posible realizar la misma búsqueda y obtener para cada uno de los resultados información en profundidad (provincia, comarca y autonomía).

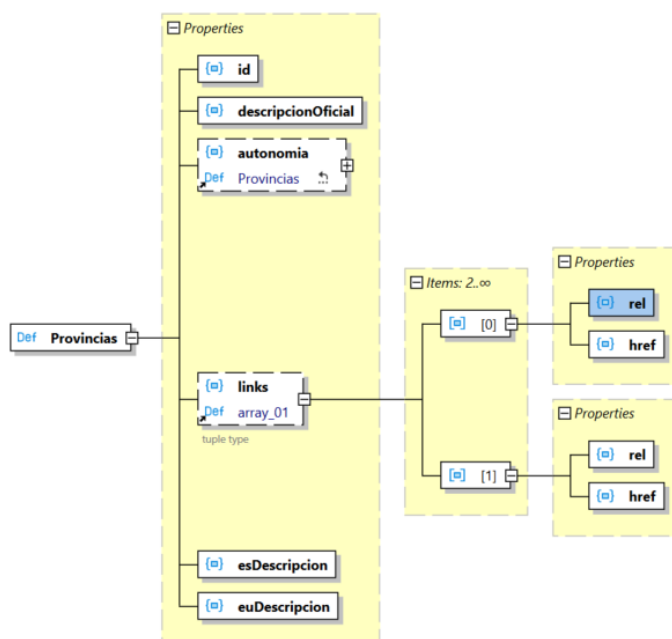
3.1.1. País



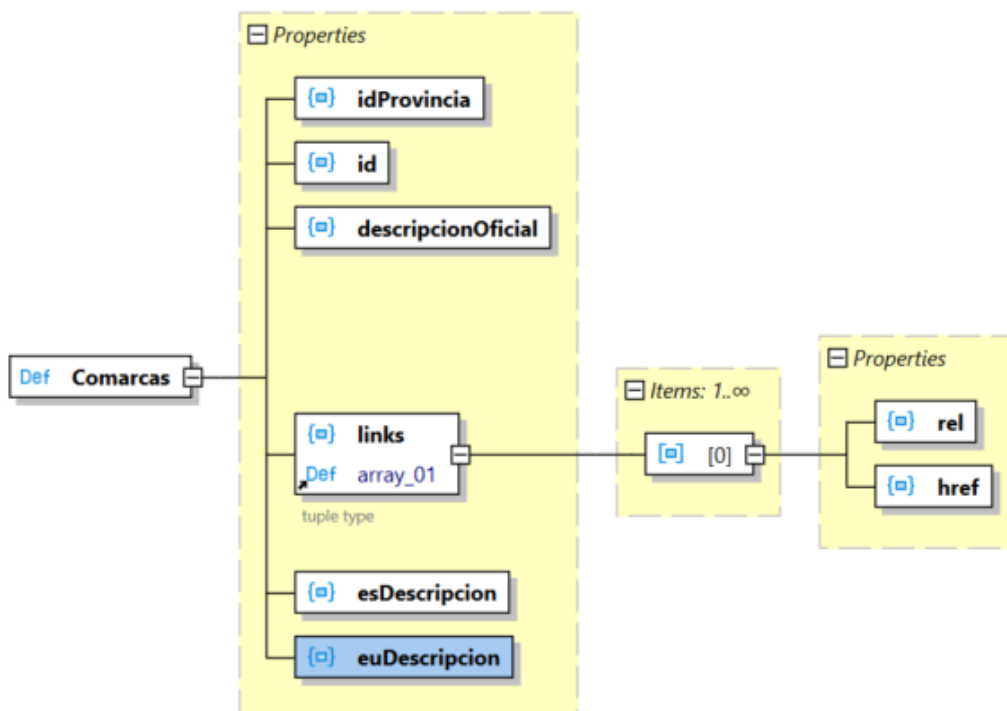
3.1.2. Autonomía



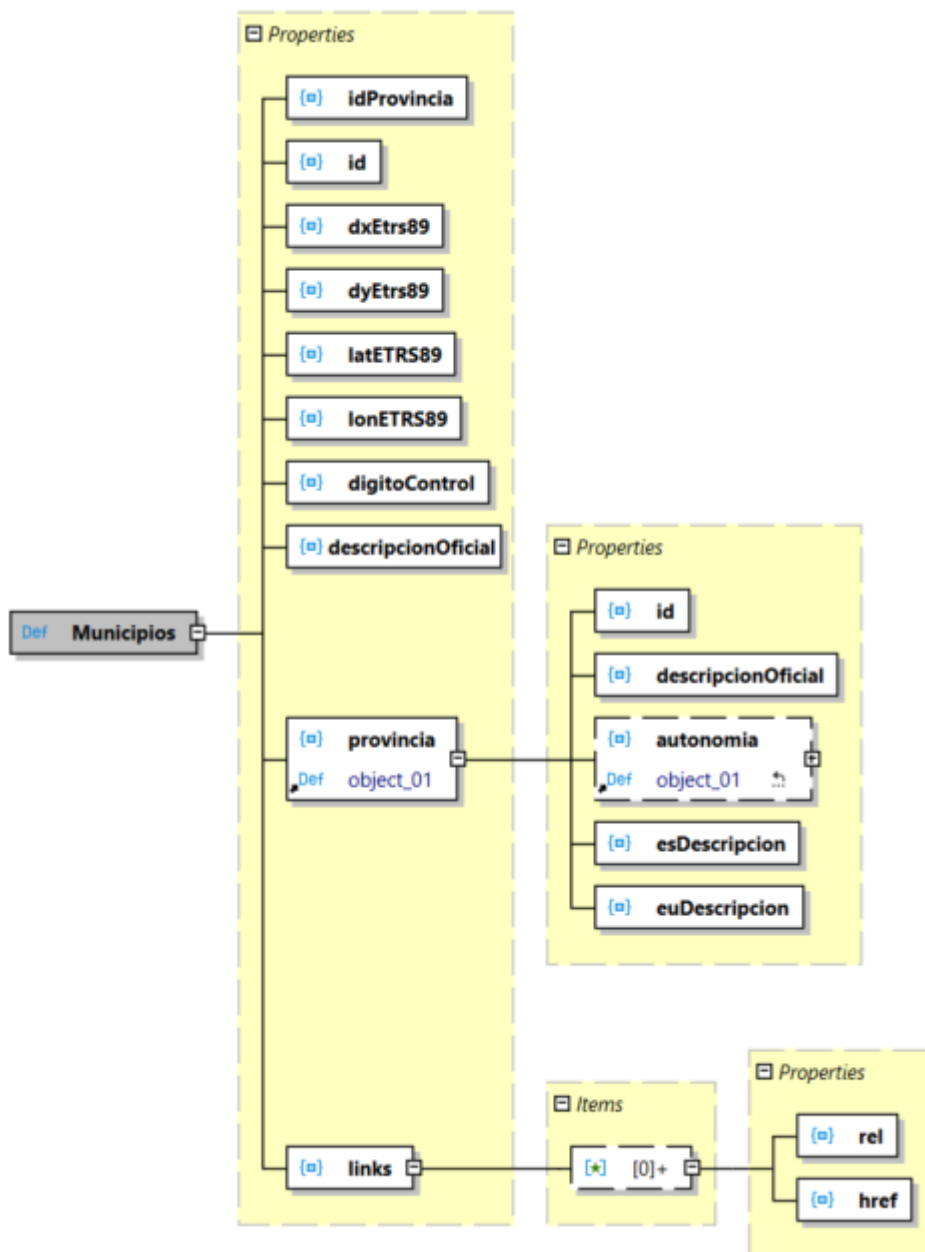
3.1.3. Provincia



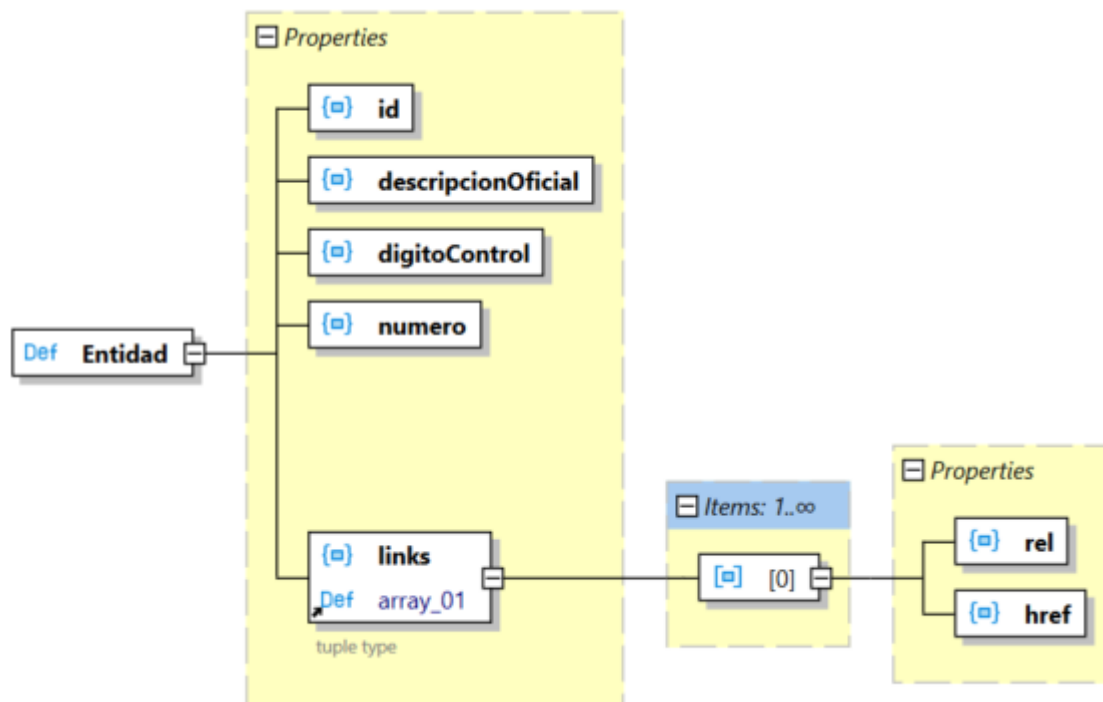
3.1.4.Comarca



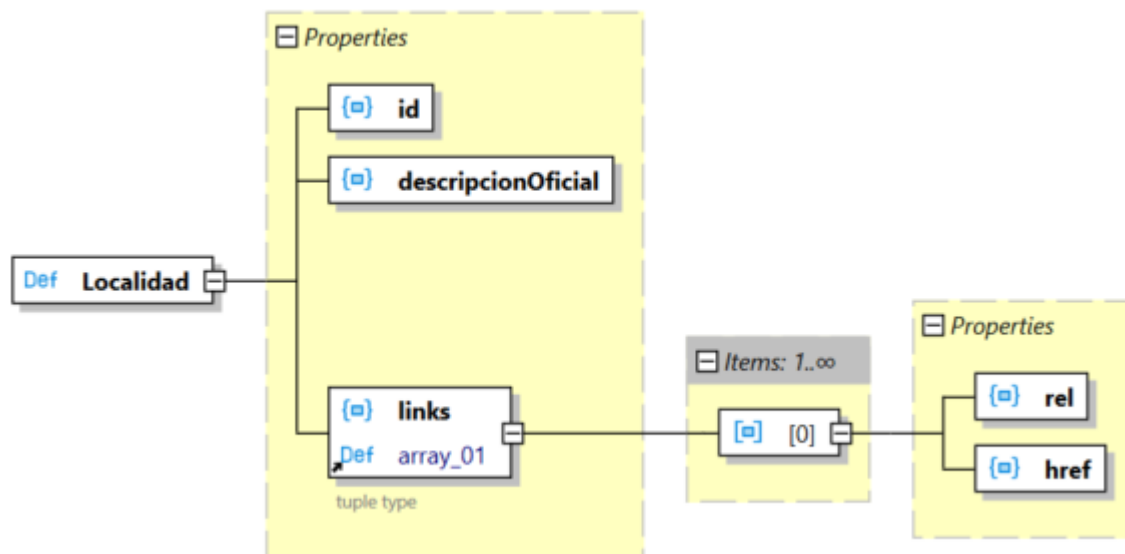
3.1.5.Municipio



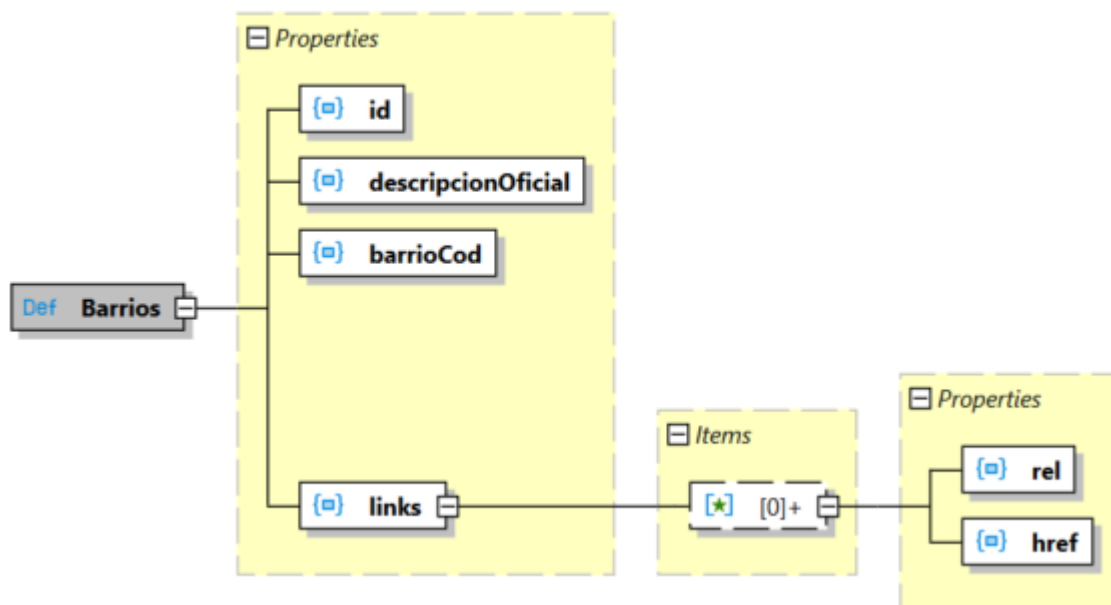
3.1.6. Entidad



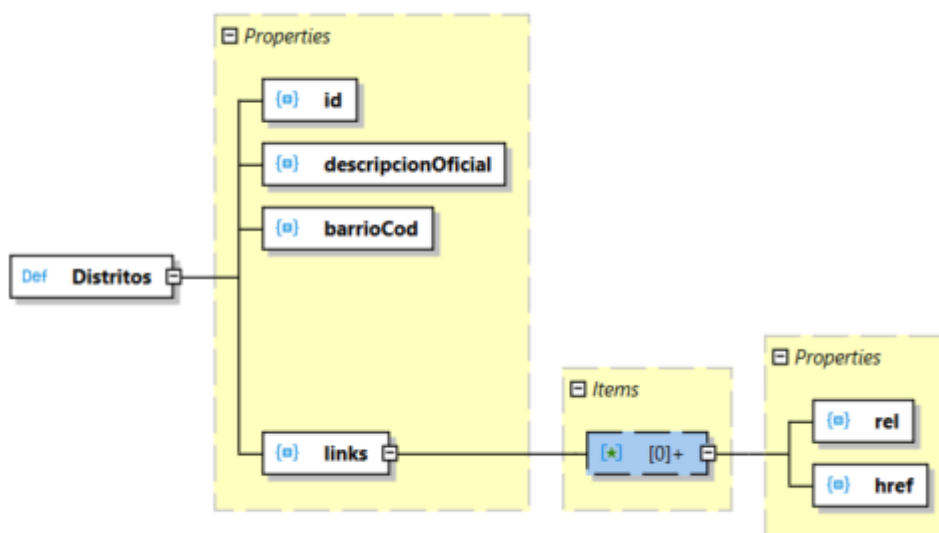
3.1.7. Localidad



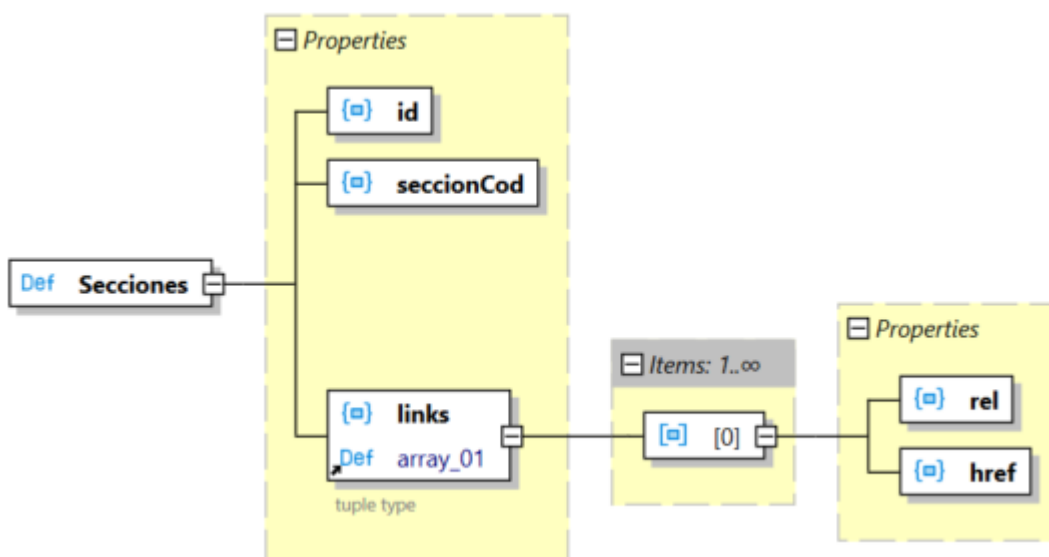
3.1.8.Barrio



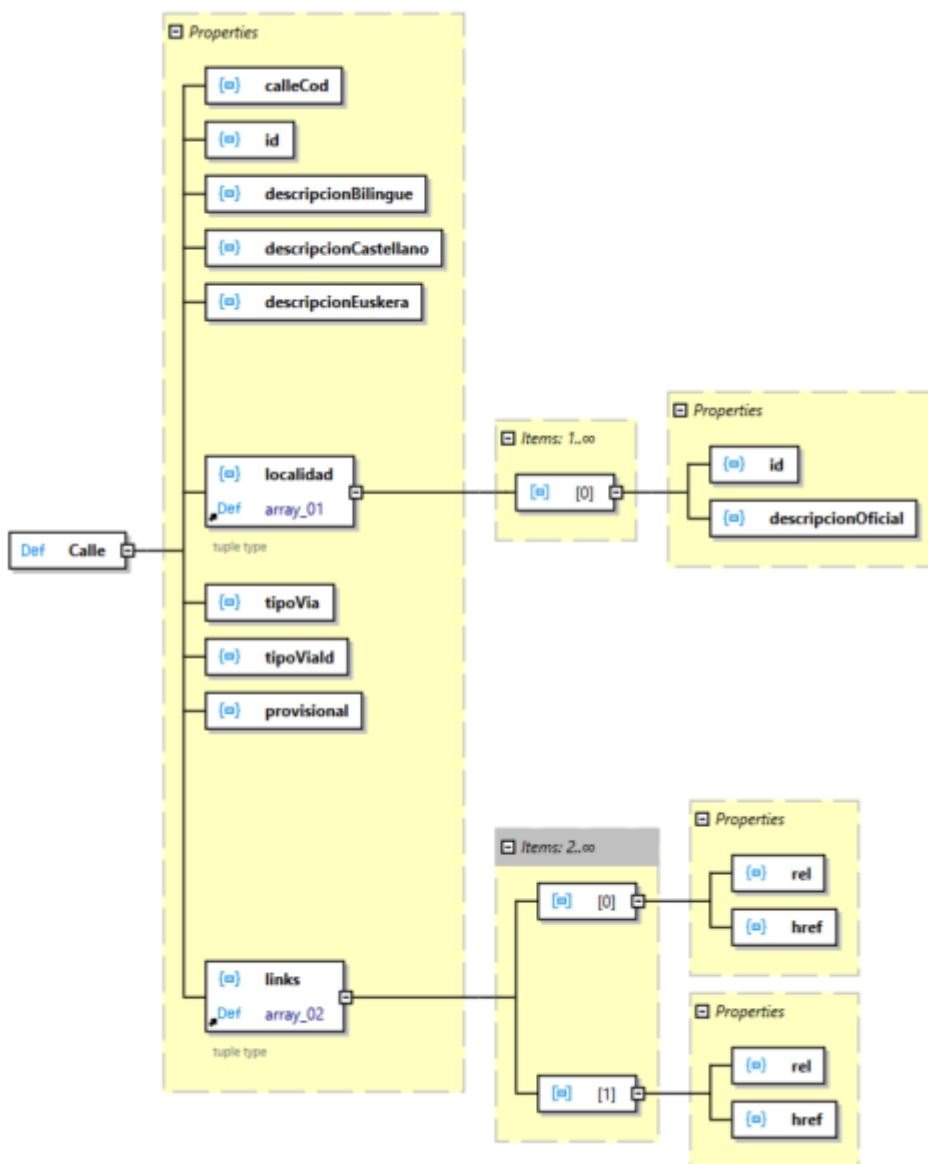
3.1.9.Distrito



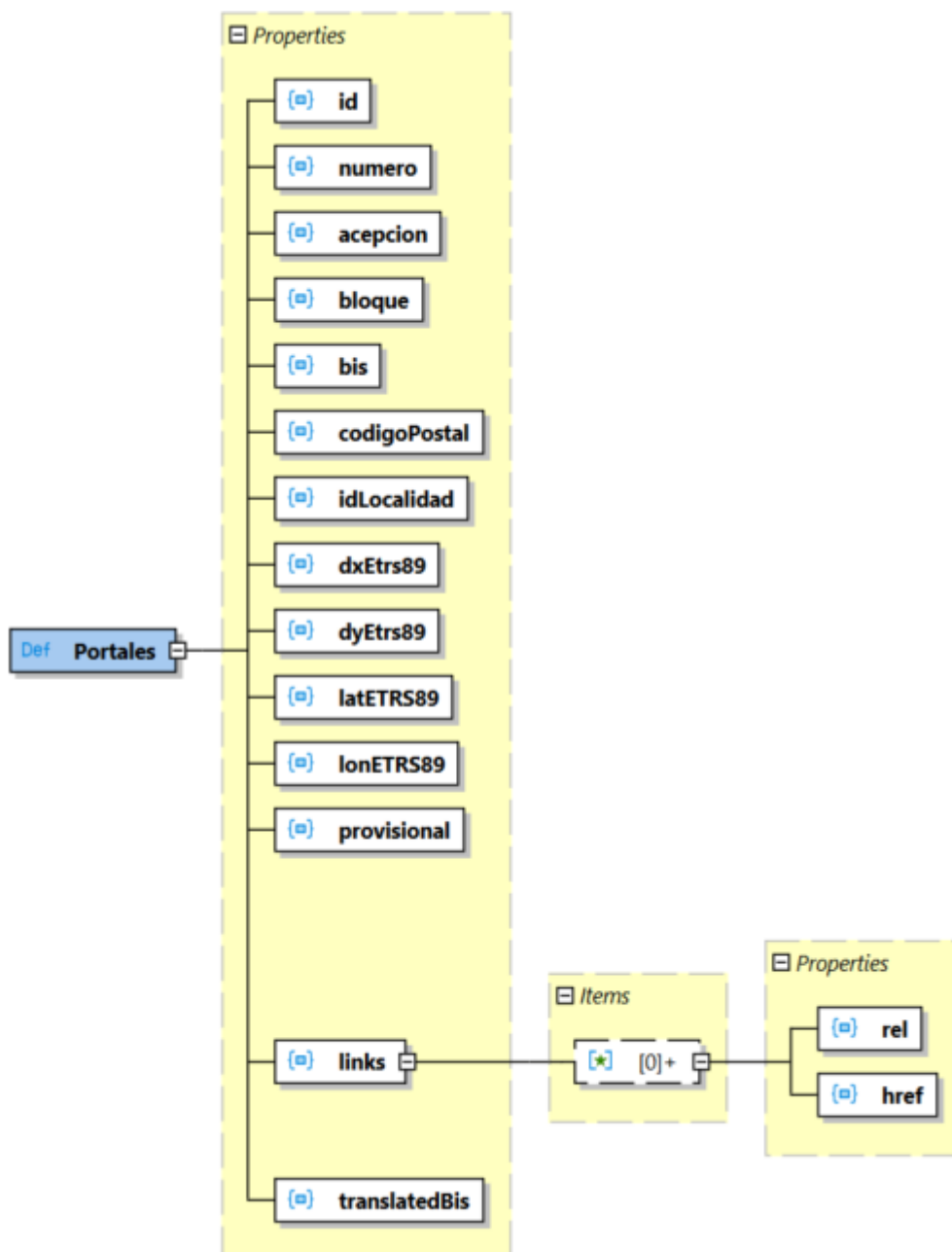
3.1.10. Sección



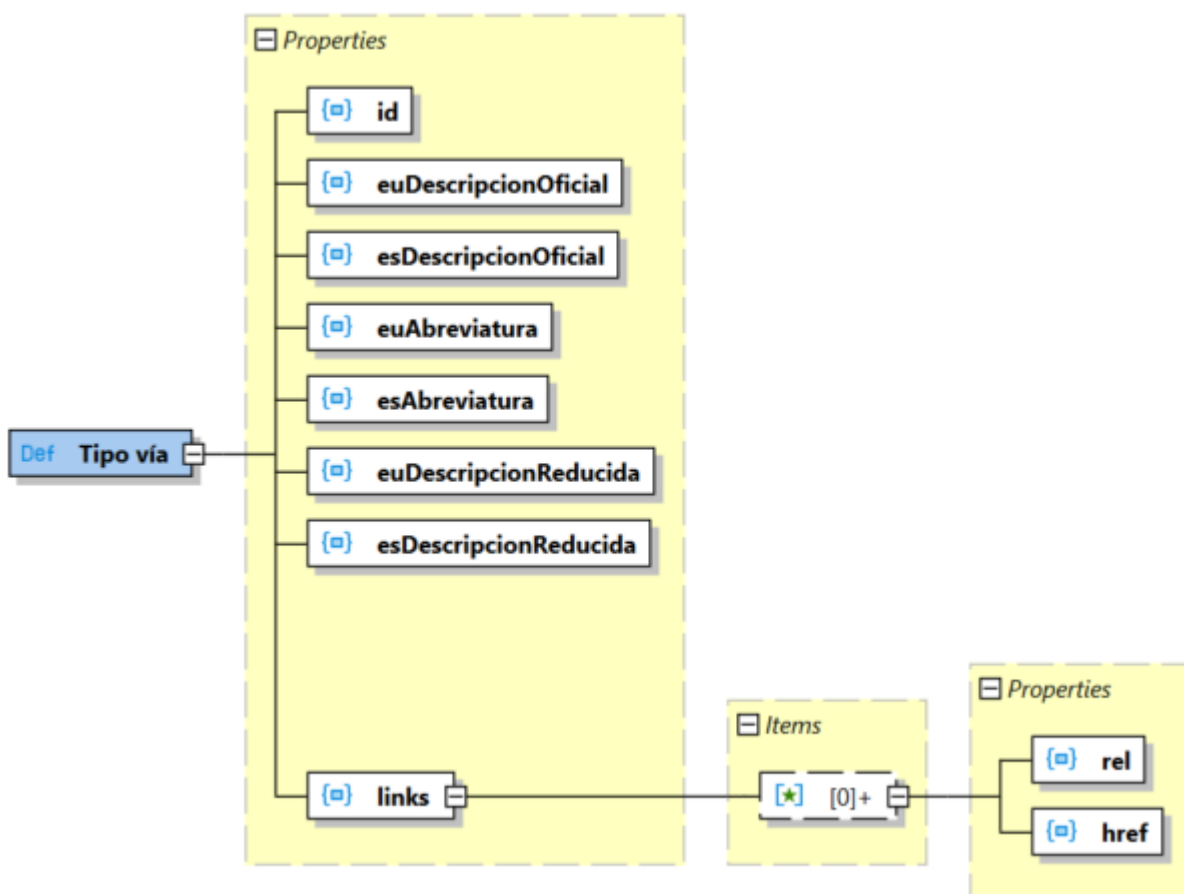
3.1.11. Calle



3.1.12. Portal



3.1.13. TipoVía



3.1.14. Edificios

